OBJECTS, ARRAYS, FUNCTIONS

OBJECTS - WHY?

```
const myFirstName = 'Ivan';
const myLastName = 'Ivanovich';
const myAge = 18;
const hasHigherEducation = true;
const isMarried = false;
const myWife = undefined;
const myMoney = null;
```

```
const IAm = {
  firstName: 'Ivan',
  lastName: 'Ivanovich',
  age: 18,
  hasHigherEducation: true,
  relationship: {
    isMarried: false,
    wife: undefined,
  },
  money: null,
};
```

OBJECT SYNTAX

Object - is a data type that can store multiple data in key-value pairs.

```
const obj = {
  key: value,
};

const obj2 = {
  key1: value1,
  key2: value2,
};
```

Each key-value pair has a colon : between them and is separated by a comma ,

JAVASCRIPT OBJECT PROPERTIES

The key-value pairs of an object are referred to as properties

```
const person = {
  name: 'John',
  age: 20,
};
```

Here, name: "John" and age: 30 are the properties of the object person.

ACCESS OBJECT PROPERTIES

You can access the value of a property by using its key.

```
const dog = {
  name: 'Rocky',
};

// access property by dot notation
console.log(dog.name); // Output: Rocky
```

```
const cat = {
  name: 'Luna',
};

// access property by bracket notation
console.log(cat['name']); // Output: Luna
```

JAVASCRIPT OBJECT OPERATIONS

MODIFY OBJECT PROPERTIES

We can modify object properties by assigning a new value to an existing key. For example,

```
const person = {
  name: 'Bobby',
  hobby: 'Dancing',
};

// modify property
person.hobby = 'Singing';

console.log(person); // Output: { name: 'Bobby', hobby: 'Singi
```

ADD OBJECT PROPERTIES

We can modify object properties by assigning a new value to an existing key. For example,

```
const student = {
  name: 'John',
  age: 20,
};

// add properties
student.rollNo = 14;
student.faculty = 'Science';

console.log(student); // Output: { name: 'John', age: 20, roll
```

DELETE OBJECT PROPERTIES

We can remove properties from an object using the delete operator. For example,

```
const employee = {
  name: 'Tony',
  position: 'Officer',
  salary: 30000,
};

// delete object property
delete employee.salary;

console.log(employee); // Output: { name: 'Tony', position: 'O
```

NESTED OBJECT CREATE

A nested object contains another object as a property. For example,

```
// outer object student
const student = {
  name: 'John',
  age: 20,

  // contains another object marks
  marks: {
    science: 70,
    math: 75,
  },
};

console.log(student); // Output: { name: 'John', age: 20, mark
```

ACCESS PROPERTIES OF NESTED OBJECTS

We can access a nested object's property using both the dot and bracket notations.

```
const student = {
  name: 'John',
  age: 20,

  marks: {
    science: 70,
    math: 75,
  },
};

// use dot notation
console.log(student.marks.science); // 70

// use bracket notation
console.log(student['marks']['math']); // 75
```

JAVASCRIPT FUNCTIONS

FUNCTION SYNTAX

A function is an independent block of code that performs a specific task

```
function name() {
  // body
  //   optional: return value;
}
```

BENEFITS OR USING FUNCTIONS

- **Reusable Code**: Since functions are independent blocks of code, you can declare a function once and use it multiple times.
- **Organized Code**: Dividing small tasks into different functions makes our code easy to organize.
- **Readability**: Functions increase readability by reducing redundancy and improving the structure of our code.

FUNCTION CALL

```
// create a function
function greet() {
  console.log('Hello World!');
}

// call the function
greet();

console.log('Outside function');
```

Output

```
Hello World!
Outside function
```

JAVASCRIPT FUNCTION ARGUMENTS

```
// function with a parameter called 'name'
function greet(name) {
  console.log(`Hello ${name}`);
}

// pass argument to the function
const res1 = greet('John');
console.log(res1); // Hello John

const res2 = greet('David');
console.log(res2); // Hello David
```

THE RETURN STATEMENT

We can return a value from a JavaScript function using the return statement.

```
// function to find square of a number
function findSquare(num) {
  // return square
  return num * num;
}

// call the function and store the result
let square = findSquare(3);

console.log(`Square: ${square}`); // Square: 9
```

THE RETURN STATEMENT TERMINATES THE FUNCTION

Any code written in the function after the return statement is not executed

```
function display() {
  console.log('This will be executed.'); // 'This will be exec

  return 'Returning from function.';

  console.log('This will not be executed.');
}

let message = display();
console.log(message); // 'Returning from function.'
```

FUNCTION EXPRESSIONS

JavaScript function expression is a way to store functions in variables

```
// store a function in the square variable
let square = function (num) {
  return num * num;
};

console.log(square(5)); // 25
```

FUNCTION RECURSION

Recursion is a technique where a function calls itself repeatedly to solve a problem

```
// recursive function
function counter(count) {
  // display count
  console.log(count);

  // condition for stopping
  if (count > 1) {
    // call counter with new value of count
    counter(count - 1);
  } else {
    // terminate execution
    return;
  }
}

// start recursion
counter(5);
```

FUNCTION RECURSION LIMIT

The recursion limit prevents errors caused by too many nested function calls.

However, the limit varies depending on the JavaScript engine and the environment in which the code is running.

For instance, the maximum limit can differ between Firefox and Chrome. Whereas, devices with higher memory might have higher recursion limits than devices with less memory.

JAVASCRIPT ARRAY

JAVASCRIPT ARRAY

An array is an object that can store multiple values at once. Arrays allow us to organize related data by grouping them within a single variable.

Instead of

```
let fruit1 = 'Apple';
let fruit2 = 'Banana';
let fruit3 = 'Orange';
```

Use

```
let fruits = ['Apple', 'Banana', 'Orange'];
```

CREATE AN ARRAY

We can create an array by placing elements inside an array literal [], separated by commas

```
const numbers = [10, 30, 40, 60, 80];

const emptyArray = []; // empty array

const dailyActivities = ['eat', 'work', 'sleep']; // array of

const mixedArray = ['work', 1, true]; // array with mixed data
```

ACCESS ELEMENTS OF AN ARRAY

Each element of an array is associated with a number called an index, which specifies its position inside the array

```
const numbers = [10, 30, 40, 60, 80];
console.log(numbers[2]); // 40
```

Code	Description
numbers[0]	Accesses the first element 10.
numbers[1]	Accesses the second element 30.
numbers[2]	Accesses the third element 40.
numbers[3]	Accesses the fourth element 60.
numbers[4]	Accesses the fifth element 80.

ADD ELEMENT TO AN ARRAY: USING THE PUSH() METHOD

The **push()** method adds an element **at the end of the array**.

```
let dailyActivities = ['eat', 'sleep'];

// add an element at the end
dailyActivities.push('exercise');

console.log(dailyActivities); // [ 'eat', 'sleep', 'exercise'
```

ADD ELEMENT TO AN ARRAY: USING THE UNSHIFT() METHOD

The **unshift()** method adds an element **at the beginning of the array**.

```
let dailyActivities = ['eat', 'sleep'];

// add an element at the beginning
dailyActivities.unshift('work');

console.log(dailyActivities); //  [ 'work', 'eat', 'sleep' ]
```

CHANGE THE ELEMENTS OF AN ARRAY

We can add or change elements by accessing the index value

```
let dailyActivities = ['eat', 'work', 'sleep'];

// change the second element
// use array index 1
dailyActivities[1] = 'exercise';

console.log(dailyActivities); // [ 'eat', 'exercise', 'sleep'
```

REMOVE ELEMENTS FROM AN ARRAY

We can remove an element from any specified index of an array using the splice() method.

```
let numbers = [1, 2, 3, 4, 5];

// remove one element starting from index 2
numbers.splice(2, 1);

console.log(numbers); //  [ 1, 2, 4, 5 ]
```

ARRAY METHODS

Method	Description
concat()	Joins two or more arrays and returns a result.
toString()	Converts an array to a string of (comma-separated) array values.
indexOf()	Searches an element of an array and returns its position (index).
find()	Returns the first value of the array element that passes a given test.
findIndex()	Returns the first index of the array element that passes a given test.
forEach()	Calls a function for each element.
includes()	Checks if an array contains a specified element.
sort()	Sorts the elements alphabetically in strings and ascending order in numbers.
slice()	Selects part of an array and returns it as a new array.
splice()	Removes or replaces existing elements and/or adds new elements.

To learn more, visit JavaScript Array Methods.

LENGTH OF AN ARRAY

We can find the length of an array using the length property

```
const dailyActivities = ['eat', 'sleep'];

console.log(dailyActivities.length); // Output: 2
```

RELATIONSHIP BETWEEN ARRAYS AND OBJECTS

In JavaScript, arrays are a type of object. However,

Arrays use numbered indexes to access elements. Objects use named indexes (keys) to access values. Since arrays are objects, the array elements are stored by reference. Hence, when we assign an array to another variable, we are just pointing to the same array in memory.

So, changing one will change the other because they're essentially the same array

```
let arr = ['h', 'e'];

let arr1 = arr; // assign arr to arr1

arr1.push('l'); // change arr1

console.log(arr); // ['h', 'e', 'l']
console.log(arr1); // ['h', 'e', 'l']
```

ITERATE THROUGH AN ARRAY: FOR LOOP

A for loop can also be used to iterate over elements of an array

```
const fruits = ['apple', 'banana', 'cherry'];

for (let i = 0; i < fruits.length; i++) {
  console.log(fruits[i]);
}
```

Output

```
apple
banana
cherry
```

USED MATERIALS

https://www.programiz.com/javascript/object

https://www.programiz.com/javascript/function

https://www.programiz.com/javascript/recursion

https://www.programiz.com/javascript/array